MCQs from exams 2021-2023

- Exponential loss: $L(w) = 1/N \sum_{i=1}^{N} exp(-y_i x_i^T w)$. If we find a vector w^* such that $L(w^*) < 1/N$, then this w^* linearly separates the dataset.
- $L_\lambda(w) = 1/N \sum_{i=1}^{N} (y_i - x_i^T w)^2 + \lambda ||w||_2)$ and $C_\lambda = min_{w \in R^d} L_\lambda(w)$. Then C_λ is a non-decreasing function of λ because of $\lambda_1 < \lambda_2, L_{\lambda_1}(w) \leq L_{\lambda_2}(w)$
- LASSO is a linear regression with L_1 regularization, it is equivalent to imposing on the weights a Laplace prior.
- Ridge regression has a larger bias and a smaller variance.
- **Logistic Regression Weights:** Positive weights ($w > 0$) increase probability of class 1; negative weights ($w < 0$) favor class 0; zero weights ($w \approx 0$) indicate irrelevant features.
- Gradient descent steps are more computationally efficient, while Newton's method takes fewer steps to converge. Newton's method is a second-order method, meaning that we must compute the Hessian of the loss (e.g. log-likelihood). This extra information makes the loss converge faster (per iteration), but it also means that we need to compute the Hessian of a very large matrix (especially when we have lots of data points)
- GANs use a discriminator and generator during training, whereas Diffusion Models only use one network that gradually removes noise from the inputs using a Markov chain. Both generate data from input noise. GANs have a dual loss function, while Diffusion Models use a simple L_2 loss.
- PCA is invariant to translation but not scaling, which is why it is important to perform standard normal scaling before applying PCA if we want to obtain meaningful results. Adding a constant feature will result in a component with zero variance that will be eliminated by PCA.
- Black-box adversarial attacks can be implemented using gradient approximation via a finite difference formula.
- Poisson, Gaussian, Bernoulli distributions are member of exponential family. Uniform and Cauchy distribution are not. The Poisson and Gaussian distributions are members of the exponential family, as seen in the lectures slides, while Cauchy and Uniform distributions are not. Indeed, the probability density function of the Cauchy and Uniform distributions cannot be written in the canonical form of the exponential family. Specifically, for the Uniform distribution, its support depends on parameter η which is not allowed by the canonical form of the exponential family.
- Nearest neighbors can be slow to find in high-dimensional spaces. Computing distances in high-dimensional spaces is expensive and thus finding nearest neighbors can be slow. (2) Nearest neighbor classifiers need to store the training data to find nearest neighbors and aggregate their predictions. (3) Other distances can be used, e.g. Manhattan distance.
- $argmin_w \ \log(p(y|x, w))$ is not equivalent to MLE. Maximizing the likelihood is the same as maximizing its log and minimizing its negative log.
- If we run Gradient Descent to solve a logistic regression task on linearly separable data, the weights will not converge. There is no closed-form solution when minimizing negative log-likelihood for logistic regression. We cannot solve for θ analytically in logistic regression like in linear regression. Optimization techniques like GD or Newton methods are required. Logistic regression finds any solution that separates two classes. To solve logistic regression, we maximize log likelihood. For linearly separable case, by increasing $||\theta||$, one could always increase the likelihood and the weights can go to infinity.
- Considering a sequence of n tokens, the computational complexity of the masked attention mechanism in BERT language models is $O(n^2)$. Masked attention is quadratic in the sequence length n because it computes the attention between all pairs of tokens.
- Consider the K-fold cross validation on a linear regression model with a sufficiently large amount of training data. When K is large, the computational complexity of the K-fold cross validation with respect to K is of order $O(K)$. Because each training process uses $(K-1)/K = 1$ fraction of the data and there are K such processes.
- The KNN algorithm need a notion of distance to asses which points are "nearest". Euclidean distance, Manhattan distance and L_4 norm can all be used in the KNN algorithm.
- What alternates in Alternating Least Squares for Matrix Factorization for a movie recommender system? updates to user embeddings and updates to movie embeddings.
- Consider a Generative Adversarial Network (GAN) which successfully produces images of goats. Which of the following statements is false? TRUE: After the training, the discriminator loss should ideally reach a constant value. TRUE: The generator aims to learn the distribution of goat images. TRUE: The generator can produce unseen images of goats. FALSE: The discriminator can be used to classify images as goat vs non-goat.

TRUE/FALSE from exams 2021-2023

- FALSE: In ridge regression, a large regularization parameter causes overfitting whereas a small regularization parameter causes underfitting. SOL: A higher value of the regularization parameter makes the model prone to choosing simple weights and thus in such a case the model underfits.
- FALSE: The loss function used in logistic regression equally penalizes positive and negative deviations from the correct lass label. SOL: Mean squared error equally penalizes deviations from the true label in the "correct"and "wrong"directions, whereas the nonlinear sigmoid function allows the logistic loss function to penalize deviation in the "wrong"direction much more heavily.
- FALSE: A model which has a high bias necessarily has a low variance. SOL: Model which outputs a total random solution with high variance has high bias and variance.
- FALSE: Generative Adversarial Networks use the generator and discriminator models during training but only the discriminator for data synthesis. SOL: Only the generator is used for synthesis.
- TRUE: Fitting a Gaussian Mixture Model with a single Gaussian (K = 1) will converge after one step of Expectation-Maximization. SOL: The E-step will assign all the points to the single Gaussian and the Gaussian parameters will be fitted to the data in a single M-step by the Maximum Likelihood Estimator.
- TRUE: Consider two fully connected networks, A and B, with a constant width for all layers, inputs and outputs. Network A has depth 3L and width H, network B has depth L and width 2H. Everything else is identical for the two networks and both L and H are large. In this case, performing a single iteration of backpropagation requires fewer scalar multiplications for network A than for network B. SOL: he number of multiplications required for backpropagation is linear in the depth and quadratic in the width, $3LH^2 < L(2H)^2 = 4LH^2$
- (CNN) FALSE: The output of a 2D convolution layer with filter size SxS where $S \geq 1$ always has a larger receptive filed than the input to the convolution. SOL: A 1x1 convolution does not increase the receptive field.
- (kNN) FALSE: Nearest neighbor classifiers cannot be used for regression because they rely on majority voting, which is not suited for continuous labels. SOL: For regression, kNNs can be used by averaging the labels of nearest neighbors, instead of majority voting.
- TRUE: (Adversarial Robustness) The primal formulation of the soft-margin SVM is NOT equivalent to ℓ_2 adversarial training for a linear model trained with the hinge loss ($\ell(z) = max\{0, 1 - z\}$).
- Text representation) FALSE: Antonyms, such as "hot"and "cold", have very different context words. SOL: Words with opposite meanings can have very similar context words.
- (Linear regression) For linear regression with no regularization, scaling the features (e.g. as in data normalization) does not change the model's performance, assuming that we can efficiently compute the optimum model in both cases, i.e. numerical stability/efficiency of finding the optimum model is not a concern. TRUE because Linear regression models are scale invariant when there is no regularization. The model will learn the same relationship between inputs and outputs regardless of the scale of the data.
- (Linear regression) For linear regression with a bias and no regularization, centering the features (as in data normalization) does not change the model's performance, assuming that we can efficiently compute the optimum model in both cases, i.e. numerical stability/efficiency of finding the optimum model is not a concern. TRUE. Linear regression models are scale invariant when there is no regularization. The model will learn the same relationship between inputs and outputs regardless of the scale of the data.
- Unnecessary polynomial expansion of input features can lead to OVERFITTING.
- (Ridge regression) Ridge regression can help mitigate the impact of ill-conditioned data matrices during training, but it does not make the solution sparse. It consists of minimizing the following loss function: $\sum_{n=1}^{N} (y_n - x_n^T W)^2 + \lambda ||w||_2^2$. TRUE. Lasso regularization makes the solution sparse.
- When training a Deep Neural Network, it is better to use classical gradient descent rather than stochastic gradient descent with mini-batches to optimize the parameters of the network. FALSE. Deep Learning requires large datasets, and going through all of the data to compute a full gradient is unnecessarily expensive, compared to many cheaper SGD steps. In addition, noisy updates for several reasons seem to perform better in the non-convex landscapes stemming from deep learning training
- One step of SGD for Logistic Regression will result in a change to the parameters only if the current example is incorrectly classified. FALSE. The gradient of logistic regression is $\Delta L(w) = (\sigma(x^T w) - y)x$, we update model parameters as long as $\sigma(x^T w)$ is not equal to $y \in \{0, 1\}$
- There are many classification problems that are not linearly separable. Tricks like kernel methods or adding a regularizer allow us to still separate the classes with a linear classifier. FALSE. Kernel method is indeed a valid trick, but not adding a regularizer. Regularizer is used to prevent overfitting, which is not the case here.
- The kernel trick allows computing inner products in infinite-dimensional feature spaces for specific choices of the kernel function. TRUE. An example is RBF kernel.
- The Expectation-Maximization (EM) algorithm is guaranteed to converge to the global maximum likelihood solution for fitting GMMs. FALSE.

TRUE/FALSE from exams 2021-2023

- When GPT models are trained on next token prediction with teacher-forcing, during training, each predicted token is added to the input sequence and fed back as input to the model in an auto-regressive manner. FALSE. In GPT models, predicting the next token in an auto-regressive manner is used for inference only, while for training the model is fed with the entire sequence of original tokens and the masked (causal) attention prevents the model from cheating.
- (Neural Networks) Weight sharing allows CNNs to deal with image data without using too many parameters. However, weight sharing increases the variance of a model. FALSE, weight sharing increases bias.
- Consider a binary classification task $L(w) = \sum_{n=1}^{N} -y_n x_n^T + log(1 + e^{x_n^T w})$ with linearly separable samples and $y \in 0, 1$. Then there exists an optimal w with exact 0 loss and 100% training accuracy. FALSE, 0 loss is not attainable.
- For any vector $v \in R^D$, let $||v||_2 = \sqrt{v_1^2 + ... + v_D^2}$ denote the Euclidean norm. For $y \in R^D, X \in R^{D \times D}$, the solution to the least squares problem: $w^* = argmin_{w \in R^D} ||y - Xw||_2^2$ is always unique. FALSE. True only if X is full rank.
- (MSE and Neural Networks) The mean squared error (MSE) is convex w.r.t the parameters of a multi layer perceptron with more than one hidden layer and linear activation function. FALSE.
- The computational complexity of a forward pass through a standard, single-headed, encoder-only transformer of depth L, embedding dimension D, sequence length S and using a batch size of N is: $O(NLSD^2)$. FALSE. The complexity is $O(NLSD^2 + NLS^2D)$ where the second term comes from the attention mechanism.
- Adversarial training typically results in a decreased (standard) accuracy on a test set drawn from the same distribution as the training and validation sets. TRUE. Robustness often comes at the cost of standard accuracy as non-robust features can be useful and generalize to the test set.

Kernel Ridge Regression and the Kernel Trick

- **Ridge Regression:**
  Objective: Minimize: $\frac{1}{2N} \sum (y_n - w^\top x_n)^2 + \frac{\lambda}{2} \|w\|^2$
  Solution: $w_* = \frac{1}{N} (\frac{1}{N} X^\top X + \lambda I_d)^{-1} X^\top y$ or equivalently: $w_* = \frac{1}{N} X^\top (\frac{1}{N} X X^\top + \lambda I_N)^{-1} y$
- **Representer Theorem:**
  Claim: For any loss ℓ and minimizer $w_* \in \arg\min_w \frac{1}{N} \sum \ell(x_n^\top w, y_n) + \frac{\lambda}{2} \|w\|^2$, there exists $a_* \in \mathbb{R}^N$ such that $w_* = X^\top a_*$.
  Meaning: Optimal w_* lies in $\text{span}\{x_1, \ldots, x_N\}$.
  Consequence: Generalizes least squares, enables kernel methods for SVM, PCA, etc.
- **Kernelized Ridge Regression:**
  Classic formulation: $w_* = \arg\min_w \frac{1}{2N} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2$
  Dual formulation: $\alpha_* = \arg\min_\alpha \frac{1}{2} \alpha^\top (\frac{1}{N} X X^\top + \lambda I_N) \alpha - \frac{1}{N} \alpha^\top y$, $w_* = X^\top \alpha_*$
  Key takeaways: - Equivalent formulations - Dual uses X only via kernel $K = XX^\top$ - Complexity depends on d, N
- **Kernel Trick:**
  Kernel function: $\kappa(x, x') = \phi(x)^\top \phi(x')$ - Directly computes similarity $\kappa(x, x')$ - Avoids explicit mapping to high-dimensional $\phi(x)$
  Purpose: Enables linear classifiers in high-dimensional spaces without direct computation in that space.
- **Predicting with Kernels:**
  Problem: $\phi(x)^\top w_*$ is expensive due to $\phi(x)$.
  Solution: Use the kernel function: $\phi(x)^\top w_* = \sum_{n=1}^{N} \kappa(x, x_n) \alpha_n$
  Remark: - $y = \phi(x)^\top w_*$: linear in feature space. - $y = f_{w_*}(x)$: nonlinear in input space.
- **Examples of Kernels:**
  1. *Linear kernel:* $\kappa(x, x') = x^\top x'$ Feature map: $\phi(x) = x$
  2. *Quadratic kernel:* $\kappa(x, x') = (xx')^2$ Feature map: $\phi(x) = x^2$
  3. *Polynomial kernel:* $\kappa(x, x') = (x^\top x')^2$ for $x, x' \in \mathbb{R}^3$ Feature map: $\phi(x) = [x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3] \in \mathbb{R}^6$
- **Building New Kernels:**
  Let κ_1, κ_2 be kernels with feature maps ϕ_1, ϕ_2.
  1. *Linear combination:* $\kappa(x, x') = \alpha\kappa_1(x, x') + \beta\kappa_2(x, x')$, $\alpha, \beta \geq 0$
  2. *Product:* $\kappa(x, x') = \kappa_1(x, x')\kappa_2(x, x')$
  Objective: Build new kernels using existing ones.
- **Mercer's Condition:**
  Question: When does a kernel $\kappa(x, x')$ correspond to a feature map ϕ such that $\kappa(x, x') = \phi(x)^\top \phi(x')$?
  Answer: Mercer's conditions: 1. $\kappa(x, x')$ is symmetric: $\kappa(x, x') = \kappa(x', x)$ 2. The kernel matrix $K = (\kappa(x_i, x_j))_{i,j=1}^N$ is positive semi-definite (psd) for all inputs.

Linear Regression

- $y^{N\times 1}$, $X^{N\times D}$, $w^{D\times 1}$
- $L(w) = \frac{1}{2N}(y-Xw)^T * (y-Xw) \Leftrightarrow \frac{1}{2N}\sum_{n=1}^{N}(y_n - x_n^T w)^2$
- $\nabla L(w) = -\frac{1}{N}X^T(y-Xw)$, complexity: $O(ND+N+ND+D)$
- The hessian: $\nabla L(w) = \frac{1}{N}X^T X$, with complexity $O(ND^2)$. It is positive semi-definite (all its eigenvalues are non-negative).
- $w^* = (X^TX)^{-1} * X^T y$. X is invertible is $\text{rank}(X) = D$. w^* has complexity $O(D^3)+O(ND^2)+O(ND)+O(D^2)$

Ridge Regression:

- $L_R(w) = \frac{1}{2N}(y-Xw)^T * (y-Xw) + \lambda||w||_2^2$
- $\nabla L_R(w) = \frac{-1}{N}X^T(y-Xw) + 2\lambda w$ and has complexity $O(ND)$
- $w_R^* = (X^TX + 2N\lambda I)^{-1}X^T y$ with complexity $O(D^3+ND^2)$
- $w_R^* = X^T(XX^T + 2N\lambda)$ **Least Squares**:
- $w^* = (X^TX)^{-1}X^T y$ and to predict: $y_m = x_m^T w^* = x_m^T(X^TX)^{-1}X^T y$

Convexity:

- A function $\mathcal{L}(\mathbf{w})$ is convex if $\mathcal{L}(\lambda\mathbf{w} + (1-\lambda)\mathbf{w}') \leq \lambda\mathcal{L}(\mathbf{w}) + (1-\lambda)\mathcal{L}(\mathbf{w}')$ for all $\lambda \in [0,1]$. **Convexity of Quadratic Function:** For $\mathcal{L}(\mathbf{w}) = \frac{1}{2N}(\mathbf{y}-\mathbf{X}\mathbf{w})^\top(\mathbf{y}-\mathbf{X}\mathbf{w})$, the Hessian $\nabla^2\mathcal{L}(\mathbf{w}) = \frac{1}{N}\mathbf{X}^\top\mathbf{X}$ is PSD, so $\mathcal{L}(\mathbf{w})$ is convex.

Generalization, Model Selection, and Validation

- **Model Selection:** Choose hyper-parameters (e.g., λ in Ridge Regression or degree d in polynomial expansion) by balancing model complexity and overfitting, often using cross-validation.
- **Generalization Error:** $\mathcal{L}_{\mathcal{D}}(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(y, f(x))]$, the expected error over $\mathcal{D}$, where ℓ is the loss function (e.g., MSE, logistic loss). Also called true risk or expected loss.
- **Empirical Error:** $\mathcal{L}_S(f) = \frac{1}{|S|}\sum_{(x_n,y_n)\in S}\ell(y_n, f(x_n))$, an unbiased estimator of the true error $\mathcal{L}_{\mathcal{D}}(f)$. Generalization gap: $|\mathcal{L}_{\mathcal{D}}(f) - \mathcal{L}_S(f)|$. When the model has been trained on the same data it is applied to, the empirical error is called the training error. Overfitting can be the reason this error is not similar to the test error.
- **Generalization Gap:** $\mathbb{P}\left[\left|\mathcal{L}_{\mathcal{D}}(f) - \mathcal{L}_{S_{\text{test}}}(f)\right| \geq \sqrt{\frac{(b-a)^2\ln(2/\delta)}{2|S_{\text{test}}|}}\right] \leq \delta$. Gap decreases as $\mathcal{O}(1/\sqrt{|S_{\text{test}}|})$.
- **Hoeffding Inequality:** $\mathbb{P}\left[\left|\frac{1}{N}\sum_{n=1}^{N}\Theta_n - \mathbb{E}[\Theta]\right| \geq \epsilon\right] \leq 2e^{-2N\epsilon^2/(b-a)^2}$, for i.i.d. $\Theta_n \in [a,b]$.
- **Hoeffding Lemma:** For X with $\mathbb{E}[X]=0$ and $X\in[a,b]$, $\mathbb{E}[e^{sX}] \leq e^{\frac{1}{8}s^2(b-a)^2}$ for any $s\geq 0$.

NNC and curse of dimensionality

- **k-NN Summary:**
  Pros: - No optimization/training - Easy to implement - Effective in low dimensions
  Cons: - Slow at query time - Poor for high dimensions - Sensitive to distance choice
- **Bias-Variance Tradeoff in k-NN** *Small k:* - Low bias (complex boundaries) - High variance (overfitting)
  Large k: - High bias (simplified prediction) - Low variance

Curse of Dimensionality:

- *Claim 1:* High $d \to$ Training set covers less space. $P(x \in \text{box}) = r^d = \alpha$. For $\alpha = 0.01$: $d=10 \to r = 0.63$, $d=100 \to r = 0.95$
- *Claim 2:* In high d, points are far apart. $P(\exists x_i \in \text{box}) \geq 1/2 \implies r \geq (1-1/2^{1/N})^{1/d}$ - For $d=10, N=500$: $r \geq 0.52$
- **Generalization Bound for 1-NN:**
  Setup: $(X,Y)\sim\mathcal{D}$ over $\mathcal{X}\times\mathcal{Y}$, $\mathcal{Y}=\{0,1\}$.
  Goal: Bound classification error: $L(f) = \mathbb{P}_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$
  Baseline: - **Bayes Classifier:** $f_*(x) = 1_{\eta(x)\geq 1/2}$, where $\eta(x) = \mathbb{P}(Y=1|X=x)$ - **Bayes Risk:** $L(f_*) = \mathbb{E}_{X\sim\mathcal{D}_X}[\min\{\eta(X), 1-\eta(X)\}]$
- **Bound on Geometric Term :**
  Given $X\sim\mathcal{D}$, let $p_k = P(X \in \text{Box}_k)$. If a box contains a neighbor in S_{train} within distance $\sqrt{d\epsilon}$, it occurs with probability: $1-(1-p_k)^N$
  Proof: Worst case: $\|X - x_i\| = \sqrt{\sum_{i=1}^{d}\epsilon^2} = \sqrt{d\epsilon}$.

Classification

- **Classification as Regression:** Classification is a regression problem with discrete labels $(x,y) \in \mathcal{X}\times\{0,1\} \subset \mathcal{X}\times\mathbb{R}$.
- The square loss we used for regression is not suitable for classification **K-NN**
- A new point x is classified based on the majority vote of its k-nearest neighbors
- **k-Nearest Neighbor (k-NN): Pros:** No training, easy to implement, works well in low dimensions. **Cons:** Slow at query time, poor for high-dimensional data, sensitive to distance metric.
- **Separating Hyperplane:** For linearly separable data, choose the hyperplane with the largest **margin**, where margin = distance from hyperplane to closest point.
- **Binary Classification:** For $(X,Y)\sim\mathcal{D}$ with $Y\in\{-1,1\}$, *0-1 Loss:* $\ell(y,y') = 1_{y\neq y'}$. **True Risk:** $L_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}}[1_{Y\neq f(X)}] = \mathbb{P}_{\mathcal{D}}[Y\neq f(X)]$. Goal: minimize $L_{\mathcal{D}}(f)$.
- **Bayes Classifier:** The classifier $f^* = \arg\min_f L_{\mathcal{D}}(f)$ achieves optimal performance. Claim: $f^*(x) \in \arg\max_{y\in\{-1,1\}}\mathbb{P}(Y=y \mid X=x)$. *Note:* It is an unattainable gold standard as $\mathcal{D}$ is unknown.
- **Empirical Risk Minimization (ERM):** Minimize $L_{\text{train}}(f) = \frac{1}{N}\sum_{n=1}^{N}1_{f(x_n)\neq y_n} = \frac{1}{N}\sum_{n=1}^{N}1_{y_n f(x_n)\leq 0}$. *Problem:* L_{train} is not convex: (1) $\mathcal{Y}$ is discrete, (2) Indicator function 1 is not continuous.

Support Vector Machines (SVM)

- **Hard-SVM: Max-margin hyperplane:** First we assume the dataset is linearly separable. The margin of hyperplane is: $min_{n\leq N}|w^\top x_n|$. The hard svm problem is these three equivalences:
- $min\frac{1}{2}\|w\|^2$ s.t. $\forall n, y_n x_n^\top w \geq 1$
- $\max_{w,\|w\|=1}\min_{n\leq N}|w^\top x_n|$ s.t. $\forall n, y_n x_n^\top w \geq 0$
- $\max_{M\in\mathbb{R},w,\|w\|=1} M$ s.t. $\forall n, y_n x_n^\top w \geq M$. $M = \frac{1}{\|w\|}$
- **Soft SVM:** Relaxation of Hard SVM for non-linearly separable data. *Idea:* Maximize margin, allow constraint violations. *How:* Add positive slack variables $\xi_1,\ldots,\xi_N$. Replace the constraints with: $y_n x_n^\top w \geq 1-\xi_n, \xi_n \geq 0$.
  Objective: $\min_{\mathbf{w},\xi}\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_{n=1}^{N}\xi_n \leftrightarrow \min_{\mathbf{w}}\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_{n=1}^{N}[1-y_n x_n^\top w)]_+$ where $[a]_+ = max\{0,a\}$ **Losses for Classification**
  Margin-based losses ($\eta = y\mathbf{x}^\top\mathbf{w}$):
  - **Quadratic Loss (MSE):** $\text{MSE}(\eta) = (1-\eta)^2$
  - **Logistic Loss:** $\text{Logistic}(\eta) = \frac{\log(1+e^{-\eta})}{\log(2)}$
  - **Hinge Loss:** $\text{Hinge}(\eta) = [1-\eta]_+$

  Common features: Convex, upper bound for zero-one loss.
  Behavioral differences:
  - **MSE:** Penalizes any deviation from 1.
  - **Logistic Loss:** Asymmetric cost; penalty always incurred.
  - **Hinge Loss:** Penalty for incorrect or low-confidence predictions.
- **Min and Max Interchange**
  Always TRUE: $\max_\alpha\min_w G(w,\alpha) \leq \min_w\max_\alpha G(w,\alpha)$. Equality if G is convex in w, concave in α and the domains of w and α are convex and compact. Proof: 1. $\min_w G(\alpha,w) \leq G(\alpha,w')\ \forall w'$ 2. $\max_\alpha\min_w G(\alpha,w) \leq \max_\alpha G(\alpha,w')$ 3. $\max_\alpha\min_w G(\alpha,w) \leq \min_{w'}\max_\alpha G(\alpha,w')$

Bias-Variance Decomposition

- **Bias-Variance Tradeoff:** Simple models (e.g., low-degree polynomials) have **large bias** (average predictions do not fit the data well) but **small variance** (predictions are stable across datasets).
- **Bias-Variance Tradeoff:** Complex models (e.g., high-degree polynomials) have **low bias** (average predictions fit the data well) but **high variance** (predictions vary significantly across datasets).
- **Data Model:** $y = f(x)+\varepsilon$, where $f(x)$ is the true model, $\varepsilon\sim\mathcal{D}_\varepsilon$ (i.i.d. noise, $\mathbb{E}[\varepsilon]=0$), and $x\sim\mathcal{D}_x$. The model is not realizable if $f(x)$ is not in our model class.
- **Error Decomposition:** For $f_S = \mathcal{A}(S)$, the expected error at x_0 is $L(f_S) = \mathbb{E}_{\varepsilon\sim\mathcal{D}_\varepsilon}\left[(f(x_0)+\varepsilon-f_S(x_0))^2\right]$. Randomness arises from the training set S.
- **Bias-Variance Decomposition:** $\mathbb{E}_{S,\varepsilon}\left[(f(x_0)+\varepsilon-f_S(x_0))^2\right] = \text{Var}[\varepsilon] + (f(x_0)-\mathbb{E}_S[f_S(x_0)])^2 + \mathbb{E}_S\left[(f_S(x_0)-\mathbb{E}_S[f_S(x_0)])^2\right]$. To minimize the true error, we must choose a method that achieves low bias and low variance simultaneously.
- **Bias:** Bias: $(f(x_0)-\mathbb{E}_S[f_S(x_0)])^2$, measures how far the expected prediction is from the true value $f(x_0)$. *Low model complexity $\to$ high bias, high complexity $\to$ low bias.*
- **Variance:** $\text{Var} = \mathbb{E}_S\left[(f_S(x_0)-\mathbb{E}_S[f_S(x_0)])^2\right]$, measures the variability of predictions at x_0 across different training sets. *High model complexity $\to$ high variance.*

Logistic Regression

- **Logistic function and properties:**
- $\sigma(\eta) = \frac{e^\eta}{1+e^\eta}$
- $1-\sigma(\eta) = \frac{1+e^\eta-e^\eta}{1+e^\eta} = (1+e^\eta)^{-1}$
- $\sigma'(\eta) = \frac{e^\eta(1+e^\eta)-e^\eta e^\eta}{(1+e^\eta)^2} = \frac{e^\eta}{(1+e^\eta)^2} = \sigma(\eta)(1-\sigma(\eta))$
- **MLE:** Maximize likelihood $w_* = \arg\max_w\prod_{n=1}^{N}p(z_n \mid w)$. Equivalent to minimizing negative log-likelihood: $w_* = \arg\min_w\sum_{n=1}^{N} -\log(p(z_n \mid w))$. Consistent under mild conditions, meaning if the data are generated according to the model, the MLE converges to the true parameter as $n\to\infty$
- The likelihood is proportional to: $\mathcal{L}(w) \propto \prod_{n=1}^{N}\sigma(x_n^\top w)^{y_n}[1-\sigma(x_n^\top w)]^{1-y_n}$
- It is more convenient to work with negative log-likelihood: **Negative Log-Likelihood:** $L(w) = argmin\ L(w) = \frac{1}{N}\sum_{n=1}^{N}\left(-y_n x_n^\top w + \log(1+e^{x_n^\top w})\right)$, where minimizing $L(w)$ is equivalent to maximizing the likelihood.
- **Gradient of Negative Log-Likelihood:** $\nabla L(w) = \frac{1}{N}\mathbf{X}^\top(\sigma(\mathbf{X}w)-\mathbf{y}) \leftrightarrow \frac{1}{N}\sum_{n=1}^{N}(\sigma(x_n^T w)-y_n)x_n$ where $\sigma(\mathbf{X}\mathbf{w}) = [\sigma(x_1^\top w),\ldots,\sigma(x_N^\top w)]^\top$ and $\mathbf{y} = [y_1,\ldots,y_N]^\top$.
- **Hessian of L (Convexity Proof):** $\nabla^2 L(w) = \frac{1}{N}\mathbf{X}^\top\mathbf{S}\mathbf{X}$, $\mathbf{S} = \text{diag}[\sigma(x_n^\top w)(1-\sigma(x_n^\top w))] \succeq 0$. Conclusion: L is convex as $\nabla^2 L(w) \geq 0$.
- **Minimizing L:**
  - **Gradient descent:** $w_{t+1} = w_t - \frac{\eta_t}{N}\sum_{n=1}^{N}(\sigma(x_n^\top w_t) - y_n)x_n$
  - **Stochastic gradient descent:** $w_{t+1} = w_t - \eta_t(\sigma(x_{n_t}^\top w_t) - y_{n_t})x_{n_t}$, $\mathbb{P}[n_t = n] = \frac{1}{N}$
  - **Newton's method:** $w_{t+1} = w_t - \eta_t\nabla^2 L(w_t)^{-1}\nabla L(w_t)$

  Note: Newton's method is faster but computationally expensive (Hessian $\nabla^2 L$).
- **Linearly Separable Data:** Add ℓ_2-regularization to stabilize optimization and prevent overfitting: $L(w) = \frac{1}{N}\sum_{n=1}^{N}(-y_n x_n^\top w + \log(1+e^{x_n^\top w})) + \frac{\lambda}{2}\|w\|_2^2$

Neural Networks

- **Single Neuron View:**
  Output of neuron j in layer l: $x_j^{(l)} = \phi\left(\sum_{i=1}^{K}x_i^{(l-1)}w_{ij}^{(l)} + b_j^{(l)}\right)$ where ϕ is a non-linear activation (essential for non-linear functions), $w_{ij}^{(l)}$ is the weight from node i (layer $l-1$) to node j (layer l) and $b_j^{(l)}$ is the bias for node j in layer l.
- **NN Training:**
  Last layer: Linear prediction: $h(x) = f(x)^\top W^{(L+1)} + b^{(L+1)}$
  ML tasks: - **Regression:** $\ell(y,h(x)) = (h(x)-y)^2$ - **Binary classification:** $\ell(y,h(x)) = \log(1+\exp(-yh(x)))$ - **Multi-class classification:** $\ell(y,h(x)) = -\log\left(\frac{\exp(h(x)_y)}{\sum_{i=1}^{K}\exp(h(x)_i)}\right)$
- **Two Sigmoids Approximate a Rectangle:**
  Function: $(x_1,x_2)\mapsto\phi(w(x_1-a_1)) - \phi(w(x_1-b_1))$
  - Rectangle: - Ranges from a_1 to b_1 in x_1. - Unbounded in x_2.
- **ℓ_∞ Approximation with PWL Functions:**
  Def: Piecewise linear (PWL) function: $q(x) = \sum_{i=1}^{m}(a_i x + b_i)1_{r_{i-1}\leq x<r_i}$, with continuity: $a_i r_i + b_i = a_{i+1}r_i + b_{i+1}$.
  ℓ_∞-Approximation (Shekhtman, 1982): For f continuous on $[c,d]$ and $\epsilon>0$, $\exists$ PWL q such that: $\sup_{x\in[c,d]}|f(x)-q(x)| \leq \epsilon$
- **NN Training with SGD:**
  Update rule: $(w_{i,j}^{(l)})_{t+1} = (w_{i,j}^{(l)})_t - \gamma\frac{\partial\mathcal{L}_n}{\partial w_{i,j}^{(l)}}$, $(b_i^{(l)})_{t+1} = (b_i^{(l)})_t - \gamma\frac{\partial\mathcal{L}_n}{\partial b_i^{(l)}}$
- **Cost Function:**
  $\mathcal{L} = \frac{1}{2N}\sum_{n=1}^{N}\left(y_n - f^{(L+1)}\circ\cdots\circ f^{(1)}(x_n)\right)^2$
  Remarks: - $\mathcal{L}$ depends on all weights $\mathbf{W}^{(l)}$, biases $\mathbf{b}^{(l)}$. - Each $f^{(l)}$ is parameterized by $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$.
  Individual loss for SGD: $\mathcal{L}_n = \frac{1}{2}\left(y_n - f^{(L+1)}\circ\cdots\circ f^{(1)}(x_n)\right)^2$
  Goal: Compute $\frac{\partial\mathcal{L}_n}{\partial w_{i,j}^{(l)}}$ and $\frac{\partial\mathcal{L}_n}{\partial b_i^{(l)}}$ for all (i,j,l).
  - $\mathcal{L}_n = \frac{1}{2}(y_n - f(x_n))^2$ (loss per sample).

  Forward Pass
  We can compute $z^{(l)}$ and $x^{(l)}$ by a forward pass in the network: $x^{(0)} = x_n \in \mathbb{R}^d$, $z^{(l)} = (\mathbf{W}^{(l)})^\top x^{(l-1)} + b^{(l)}$, $x^{(l)} = \phi(z^{(l)})$ One pass over the network: $\mathcal{O}(K^2 L)$.

Convolutional Nets

- **Convolution:** For a filter f of size K and stride S, $x^{(1)}_{n,m} = \sum_{k=0}^{K-1}\sum_{l=0}^{K-1} f_{k,l}\, x^{(0)}_{nS+k,mS+l}$.
  Example with $K=3, S=1$: $x^{(1)}_{0,0} = \sum_{k=0}^{2}\sum_{l=0}^{2} f_{k,l}x_{k,l}$, $x^{(1)}_{0,1} = \sum_{k=0}^{2}\sum_{l=0}^{2} f_{k,l}x_{k,1+l}$.
  Filters are learnable weights shared across all positions.
- **Size correspondence in Conv Layer:**
  Input size $(W_1, H_1) = (5,5)$, Filter size $F=3$, Stride $S=2$, Padding $P=1$.
  Output size: $W_2 = \frac{W_1 - F + 2P}{S} + 1 = \frac{5-3+2}{2} + 1 = 3$, $H_2 = \frac{H_1 - F + 2P}{S} + 1 = \frac{5-3+2}{2} + 1 = 3$. Output size is $(W_2, H_2) = (3,3)$.
- **Weight Decay: ℓ_2-regularization**
  Objective: $\min \mathcal{L} + \frac{\lambda}{2}\sum_l \|\mathbf{W}^{(l)}\|_F^2$.
  Weight decay helps generalization by favoring small weights.
  Gradient Descent Update: $w^{(l)}_{i,j} \leftarrow (1-\eta\lambda)w^{(l)}_{i,j} - \eta\nabla\mathcal{L}$
  Interaction with BatchNorm: 1.BN($\mathbf{WX}$) = BN($\alpha\mathbf{WX}$) for $\alpha > 0$. 2. $\mathbf{W}$ is scale-invariant; no direct regularization effect.
- **Dropout Results:** Dropout improves generalization by reducing test error, especially in fully connected networks (e.g., on MNIST). However, it often requires more training iterations to converge due to added stochastic noise.

Transformers

- **Transformer:** A neural network f that maps a sequence to another sequence using self-attention to mix information across sequence elements.
- **Transformer Architecture:** Converts input sequence into tokens (e.g., one-hot vectors, flattened patches). A sequence of L transformer blocks applies self-attention and MLP sub-blocks to transform tokens into D-dimensional vectors. The output transformation maps vectors to the desired output format (e.g., classification, sequences).
- **Transformer Block:** Consists of Self-Attention (SA) for mixing information between tokens and a Multi-Layer Perceptron (MLP) for mixing information within tokens. Includes skip connections and Layer Normalization (LN) applied before SA and MLP.
- **Self-Attention:** Maps tokens (real-valued vectors) to tokens via a learned input-dependent weighted average. Given input $X \in \mathbb{R}^{T\times D}$, compute: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. $z = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}K}\right)V$ where $W_Q, W_K, W_V \in \mathbb{R}^{D\times Dk}$ are parameters, and softmax$(\cdot)$ is row-wise. Complexity: $O(T^2D)$.
- **Mixing within Tokens:** An MLP applies the same transformation to each token: MLP$(X) = \phi(XW_1)W_2$ where $W_1, W_2 \in \mathbb{R}^{D\times D}$ are learned, ϕ is a nonlinearity (e.g., ReLU or GeLU), and bias terms may be included. Each token is processed independently.
- **Transformer Variants:** 1. **Encoders** (e.g., classification): Produce fixed-size outputs, process inputs simultaneously. 2. **Decoders** (e.g., ChatGPT): Autoregressively generate tokens $x_{t+1} \sim \text{softmax}(f(x_1,\ldots,x_t))$, enabling responses of arbitrary length. 3. **Encoder-Decoder** (e.g., translation): Encode the input (e.g., one language) and decode to the output (e.g., another language) token by token.

Adversarial Machine Learning

- **Standard vs. Adversarial Risk: Classification Problem:** $(X,Y)\sim\mathcal{D}$, $Y\in\{-1,1\}$ **Standard Risk:** $\mathcal{R}(f) = \mathbb{E}_{\mathcal{D}}\left[1_{f(X)\neq Y}\right] = \mathbb{P}_{\mathcal{D}}[f(X)\neq Y]$ **Adversarial Risk:** $\mathcal{R}_\epsilon(f) = \mathbb{E}_{\mathcal{D}}\left[\max_{\hat{X},\|\hat{X}-X\|\leq\epsilon} 1_{f(\hat{X})\neq Y}\right]$
- **Adversarial Examples:** Maximize loss w.r.t. inputs: $\max_{\hat{x},\|\hat{x}-x\|\leq\epsilon} 1_{f(\hat{x})\neq y}$.
- **White-Box Adversarial Examples:** Solve $\max_{\hat{x},\|\hat{x}-x\|\leq\epsilon}\ell(yg(\hat{x}))$.
  Gradient: $\nabla_x\ell(yg(x)) = y\ell'(yg(x))\nabla_x g(x) \leq 0$. Move in direction $\propto -y\nabla_x g(x)$.
  Interpretation: $f(x) = \text{sign}(g(x))$.
  - If $y=1$: decrease $g(x)$, follow $-\nabla_x g(x)$.
  - If $y=-1$: increase $g(x)$, follow $\nabla_x g(x)$.

  Why ℓ? Directly minimizing $yg(\hat{x})$ fails for multi-class and robust training.
- **Black-Box Attacks: Summary**
  - Query-based methods require many queries (10k–100k), especially decision-based → easy to restrict access.
  - Surrogate model $\hat{f}$ is costly and success is not guaranteed.
  - Key challenge: implementation of *physically realizable attacks.*

Ethics and Fairness in ML

- **From Data to Models: Disparities Can Be Preserved** Training data contains:**Knowledge:** Patterns we want to learn. **Stereotypes:** Patterns we want to avoid learning.
- ML algorithms extract both unless explicitly guided.
- Removing attributes like gender is insufficient due to **redundant encodings** (e.g., correlated attributes).
- Redundant encodings may be relevant to the task.
- **Disparities Can Be Introduced:** Uniform subsampling reduces data for minorities, especially if they are already underrepresented. ML relies on large datasets, so fewer samples harm performance for minorities. Average true error hides poor minority performance. This is worse for anomaly detection (e.g., Nymwars). Conclusion: ML generalizes based on majority culture, leading to high error for minorities to avoid overfitting.
- **Decreasing Selection Disparity:** GPA is a proxy for demographic attributes; removing it reduces model accuracy. Use group-specific cutoffs for equal hiring probability, but this may create unfair outcomes. Alternatively, reduce GPA weight and increase diversity in selection.
- **Sensitive Attributes:** Features X can encode sensitive attributes. Introducing a variable A for protected class membership highlights that removing sensitive attributes does not ensure fairness. Correlated features can reconstruct the attribute, leading to redundant encodings and equivalent classifiers.
- **Three Fundamental Fairness Criteria:** Equalize statistical quantities involving group membership A (1960s, Anne Cleary). Fairness criteria relate to (A,Y,R): Independence: R independent of A. Separation: R independent of A given Y. Sufficiency: Y independent of A given R.
- **Can We Satisfy Them Simultaneously?** Independence: R independent of A $\Rightarrow$ equal acceptance rates. Separation: R independent of A given Y $\Rightarrow$ equal error rates. Sufficiency: Y independent of A given R $\Rightarrow$ calibration by group. Informal theorem: these criteria are mutually exclusive except in degenerate cases.

Text Representation Learning

- **Motivation:** Represent text numerically (word embeddings) for ML tasks. Use co-occurrence matrices to capture context.
- **Matrix Factorization:** Decompose co-occurrence matrix to learn embeddings.
- **GloVe:** Combines co-occurrence statistics with matrix factorization. Loss: $\sum_{i,j} f(P_{ij})(\mathbf{w}_i^\top\mathbf{w}_j + b_i + b_j - \log P_{ij})^2$
- **Word2Vec:** Learn embeddings via CBOW (predict word from context) or Skip-gram (predict context from word). Objective: $\max\prod_{w,c\in\text{context}} P(c|w)$.
- **FastText:** Extends Word2Vec by learning subword embeddings to handle rare words.

LLMs

- **Language Modeling:** Model probability distribution over sequences of text.
  Example: $p(\text{I saw a cat on a mat}) = p(x_1,\ldots,x_t)$.
  Simplest Factorization: Predict the next word. $p(x_t|x_1,\ldots,x_{t-1})\cdot p(x_{t-1}|x_1,\ldots,x_{t-2})\cdots p(x_1)$
  Example: $p(\text{mat}|\text{I saw a cat on a}) = 0.002$.
- **Tokenization**: Split the input text into a sequence of input tokens (typically word fragments + some special symbols) according to some predefined tokenizer. The tokenizer is independent of the model training and is trained beforehand for a fixed vocabulary.

Supervised Learning

- **Supervised Learning:** Given labeled data $\mathcal{D} = \{(x_n, y_n)\}_{n\in[N]}$, where labeled data is costly and time-consuming to obtain, but unlabeled data is abundant. The **training objective** aligns model predictions $\hat{y}$ with labels y, typically via a loss function loss$(y,\hat{y})$.
- **Unsupervised Learning:** Given unlabeled data $\mathcal{D} = \{x_n\}_{n\in[N]}$. Goals include learning compact data representations, density estimation, and generative modeling. Useful for exploratory analysis, anomaly detection, feature learning, and generating new observations.
- **Clustering:** A technique for exploratory data analysis. Groups objects such that similar ones are in the same cluster and dissimilar ones are in different clusters. Some challenges in clustering are the ambiguity of what is a cluster and the lack of ground truth which makes it difficult to evaluate the success.
- **Approaches for Clustering:** Graph-based: Partition graphs to minimize inter-cluster weights (e.g., spectral clustering). Hierarchical: Build a tree of data relationships (e.g., linkage-based methods). Model-based: Use models to infer clusters (e.g., K-means, GMMs).
- **K-Means:** An unsupervised learning algorithm for clustering. Partitions data into K clusters, assigning each point to the cluster with the nearest mean. **Key Concepts:** Clusters group similar data points; centroids represent the mean position of points in a cluster. **Input:** Dataset $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$ with N datapoints, number of clusters K. **Output:** Cluster centroids $\boldsymbol{\mu}_k$, cluster assignments $z_{nk} \in \{0,1\}$. **Objective:** Minimize Within-Cluster Sum of Squares (WCSS):

$$\min_{\mathbf{z},\boldsymbol{\mu}} \mathcal{L}(\mathbf{z},\boldsymbol{\mu}) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk}\|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2^2,$$

  subject to $\boldsymbol{\mu}_k \in \mathbb{R}^D$, $z_{nk}\in\{0,1\}$, $\sum_{k=1}^K z_{nk} = 1$.
  The objective function is **non-convex** and there are multiple local minima. Some of the optimization variables are discrete.
- **K-Means Algorithm: Input:** Dataset $\mathcal{D}$, number of clusters K. **Initialize:** Randomly choose initial centroids $\boldsymbol{\mu}_1,\ldots,\boldsymbol{\mu}_K$. **Repeat until convergence:**
  1. **Assignment step:** $\forall n\in[N]$, $z_{nk} = \begin{cases}1 & \text{if } k = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|_2^2,\\ 0 & \text{otherwise.}\end{cases}$
  2. **Update step:** $\forall k\in[K]$, $\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N z_{nk}\mathbf{x}_n}{\sum_{n=1}^N z_{nk}}$.

  Iteration cost: $O(NKd)$. Each iterations of the K-means algorithm decreases the K-means objective function.
- **Convergence of K-Means:** K-means is a coordinate descent algorithm: $\mathbf{z}^{(t+1)} = \arg\min_{\mathbf{z}}\mathcal{L}(\mathbf{z},\boldsymbol{\mu}^{(t)})$, $\boldsymbol{\mu}^{(t+1)} = \arg\min_{\boldsymbol{\mu}}\mathcal{L}(\mathbf{z}^{(t+1)},\boldsymbol{\mu})$.
  Key Points: - Converges in a finite number of steps due to monotonic decrease of a lower-bounded objective. - No guarantee on the number of iterations (can be exponential). - No lower bound on the gap between the final objective and the global minimum. - May converge to a local minimum instead of the global optimum. **K-Means as Matrix Factorization:** K-Means minimizes:

$$\min_{\mathbf{z},\boldsymbol{\mu}} \mathcal{L}(\mathbf{z},\boldsymbol{\mu}) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{nk}\|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2^2 = \|\mathbf{X}^\top - \mathbf{M}\mathbf{Z}^\top\|_F^2.$$

  K-Means++: An improved initialization method for the K-means that helps to avoid poor initial centroid placements by spreading out initial centroids so the represent better data distribution. 1. Select the first centroid $\boldsymbol{\mu}_1$ randomly from $\mathcal{D}$. 2. For subsequent centroids: a. Compute $D(\mathbf{x}) = \min_{j\in\{1,\ldots,m\}}\|\mathbf{x} - \boldsymbol{\mu}_j\|_2^2$. b. Select the next centroid $\boldsymbol{\mu}_{m+1}$ with probability $P(\mathbf{x}) = \frac{D(\mathbf{x})}{\sum_{\mathbf{x}\in\mathcal{D}} D(\mathbf{x})}$.
  3. Repeat Step 2 until K centroids are chosen. 4. Run standard K-Means.
  Challenges in K-Means:
  1. Performance strongly depends on the initialization. Can be avoided by using multiple random restarts or seeding with K-means++
  2. How to define the number of clusters? Diminishing returns: A new cluster significantly reduces loss when there are too few clusters, but offers little improvement once the optimal number is reached
  3. It cannot model well clusters of arbitrary shape. K-means forces the clusters to be spherical (e.g., not elliptical) and of equal size.
  4. Sensitivity to outliers: Outliers can drag the centroids away from the actual dense areas, causing clusters to become stretched or less representative of the majority of data.
  5. Hard assignments in overlapping clusters. points

  Failures of K-means: K-means fails when clusters are not separated by the bisectors of the lines connecting their centers. **Gaussian Mixture Models:** Data is modeled as a mixture of K Gaussian distributions. **Instance space:** $\mathcal{X} = \mathbb{R}^D$.
  1. Choose a random component $k\in\{1,2,\ldots,K\}$, with $\mathbb{P}(Z=k) = \pi_k$. 2. Sample $\mathbf{x}$ from $\mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k)$:

$$p(\mathbf{x} \mid Z=k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}}\exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top\Sigma_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\right).$$

Expectation Maximization (EM) Algorithm

- **Gaussian Mixture Models:** The distribution of $\mathbf{x}$ is:

$$p(\mathbf{x}) = \sum_{k=1}^{K} \frac{\pi_k}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \Sigma_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\right).$$

  Z is a latent (hidden) variable enabling the parametric representation of $p(\mathbf{x})$.
- **MLE Objective for Gaussian Mixtures:**

$$\max_{\theta} \sum_{n=1}^{N} \log\left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \Sigma_k)\right)$$

  Key Points: - Non-convex objective. - No unique optimum (permutation of K clusters). - Objective is unbounded. - Can use black-box optimization or the EM algorithm.
- **MLE with Access to Labels for GMM:** Assume observing complete data $\mathcal{D}_c = \{(\mathbf{x}_n, z_n)\}_{n=1}^N$. **Complete log-likelihood:**

$$\mathcal{L}_c(\theta \mid \mathcal{D}_c) = \sum_{n=1}^{N} \log\left(\pi_{z_n} \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_{z_n}, \Sigma_{z_n})\right) \quad \text{(Convex!)}.$$

  MLE Closed Form:

$$\hat{\pi}_k = \frac{1}{N}\sum_n \mathbb{1}\{z_n = k\}, \quad \hat{\boldsymbol{\mu}}_k = \frac{\sum_n \mathbb{1}\{z_n = k\}\mathbf{x}_n}{\sum_n \mathbb{1}\{z_n = k\}},$$

$$\hat{\Sigma}_k = \frac{1}{\sum_n \mathbb{1}\{z_n = k\}} \sum_n \mathbb{1}\{z_n = k\}(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_k)^\top.$$

  Insight: If labels were available, the problem simplifies significantly.
- **Comparison of EM and K-Means:** - EM requires more iterations to converge compared to K-Means. - Each EM iteration is more expensive ($O(NKD^3)$ vs. $O(NKD)$). - K-Means is often used to initialize EM for faster convergence. - EM outperforms K-Means for: - Anisotropic clusters. - Unequal cluster variances. - Unevenly sized clusters. **Conclusion:** EM is more flexible but slower and more computationally expensive, making it suitable for complex clustering tasks.
- **EM Algorithm in General: Assumption:** $(\mathbf{x}, \mathbf{z})$ are random variables, where $\mathbf{x}$ is observed, and $\mathbf{z}$ (e.g., cluster membership) is not observed. Their joint distribution is $p(\mathbf{x}, \mathbf{z} \mid \theta)$.
  Goal: Maximize the log-likelihood of the observed data $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$:

$$\mathcal{L}(\theta) = \log p(\mathbf{X} \mid \theta) = \log\left(\sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z} \mid \theta)\right) = \sum_{n=1}^{N} \log \sum_{z_n} p(\mathbf{x}_n, z_n \mid \theta).$$

  Applications: Gaussian Mixture Models (GMM), Hidden Markov Models (HMM), and more.
- **EM Convergence Properties:** - It is an ascent algorithm: $\mathcal{L}(\theta^{(t)}) \leq \mathcal{L}(\theta^{(t+1)})$. - Log-likelihood sequence converges under suitable conditions. - Converges to a stationary point, not a global maximum (non-convex problem). - Similar to K-means, multiple runs are needed to identify the best solution.